

Lab Assignment-2

Aim: Use Wireshark to capture network traffic and analyze headers for different layers.

What Is Wireshark?

Wireshark is an open-source network protocol analyzer widely used to troubleshoot network issues, analyze network packets, and perform security auditing. However, its true power comes from its unmatched support for over 3000 network protocols. This lets the tool transform bytes on a network into visual network packets that you can efficiently analyze using its graphical interface and various features.

[Learn how to install Wireshark on Windows, macOS, and Linux here.](#)

Wireshark features:

- Packet capture: Wireshark can capture traffic using your network interface.
- Protocol analysis: Wireshark supports the decoding and analyzing of over 3000 network protocols so you can understand their structure and content.
- Packet filtering: Wireshark includes powerful display and capture filters that filter network traffic.
- Network packet reconstruction: Wireshark can reconstruct network packets to display application-level protocols so you can see web pages, images, or other application data.
- Network traffic statistics and visualizations: Wireshark provides statistical data and visualizations of network traffic.
- Customization: You add functionality or customization to Wireshark through its plugins.

Analyzing network traffic is crucial in cyber security. It allows you to perform blue team activities (such as detecting a cyber attack, analyzing malware, and collecting forensic evidence) and red team tasks (like stealing sensitive information, extracting files, and testing your malware's network communications).

When Should Wireshark Be Used?

Wireshark should be used whenever you need to analyze network packets and understand the network communication happening across a network.

However, its most powerful feature is its ability to perform network protocol analysis. The tool supports many network protocols and can transform network packets into humanreadable data. Using Wireshark's built-in features, you can perform statistical analysis on this data, follow TCP streams, read sensitive information, extract full files, and much more.

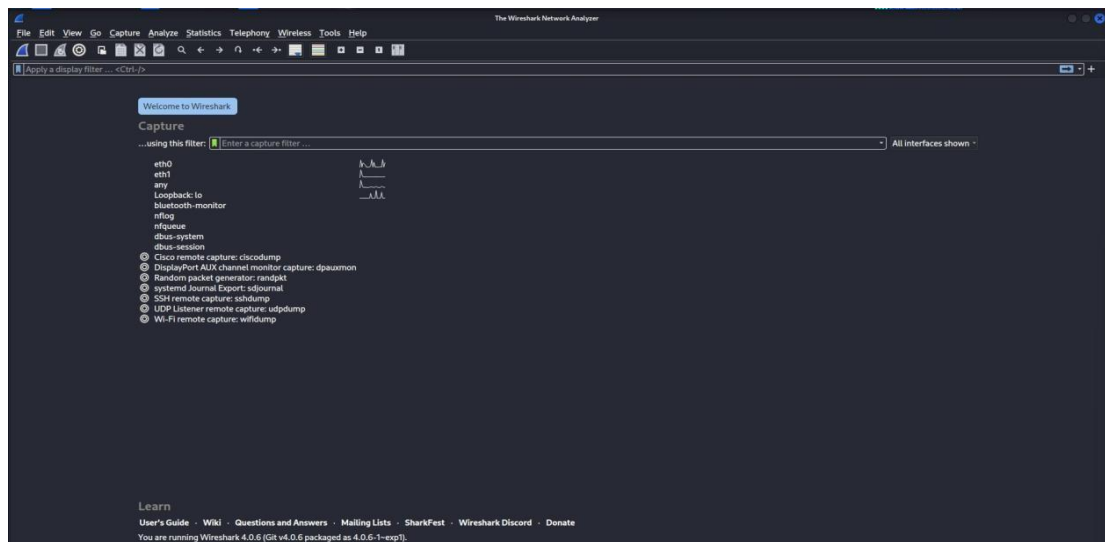
Capturing Network Traffic

To use Wireshark, you can capture network traffic or upload a packet capture (pcap) file. Let's look at how to capture network traffic.

Wireshark comes pre-installed on Kali Linux, so to capture network traffic in Wireshark, you only need to perform three steps:

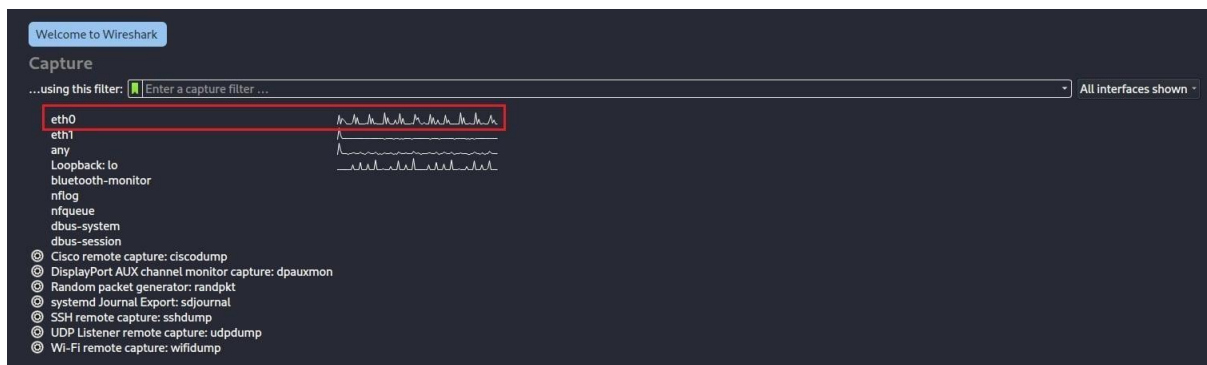
Step 1: Open the Wireshark application

First, open the Wireshark application by clicking on the icon. Wireshark's welcome screen will greet you.



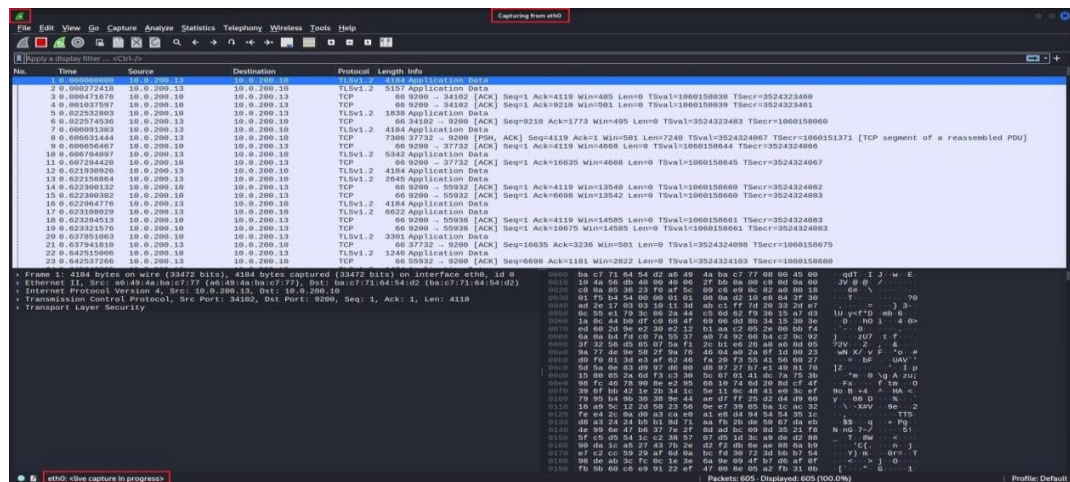
Step 2: Select a network interface to capture traffic on

From the welcome screen, select which of your network interfaces you want to capture traffic on. The line to the right of the interface indicates the network traffic flow passing through that interface.

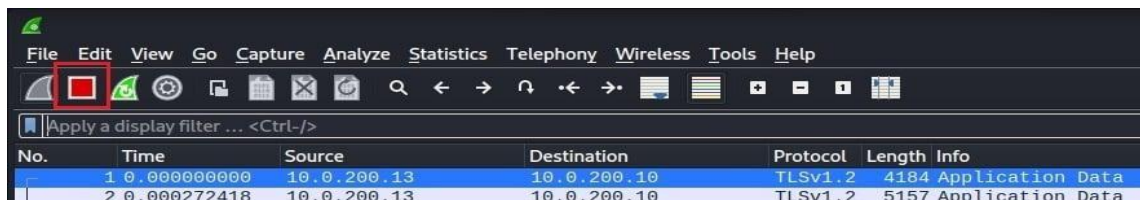


Step 3: Stopping the packet capture

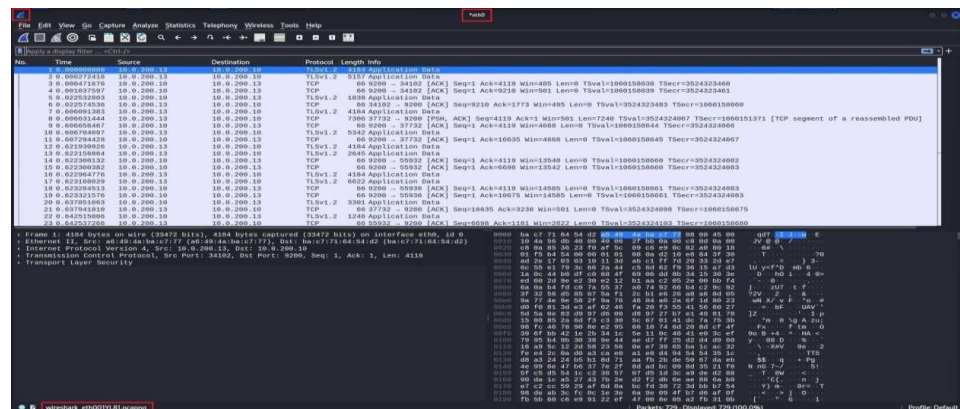
Double-clicking on the capture interface you want to capture traffic to start capturing packets.



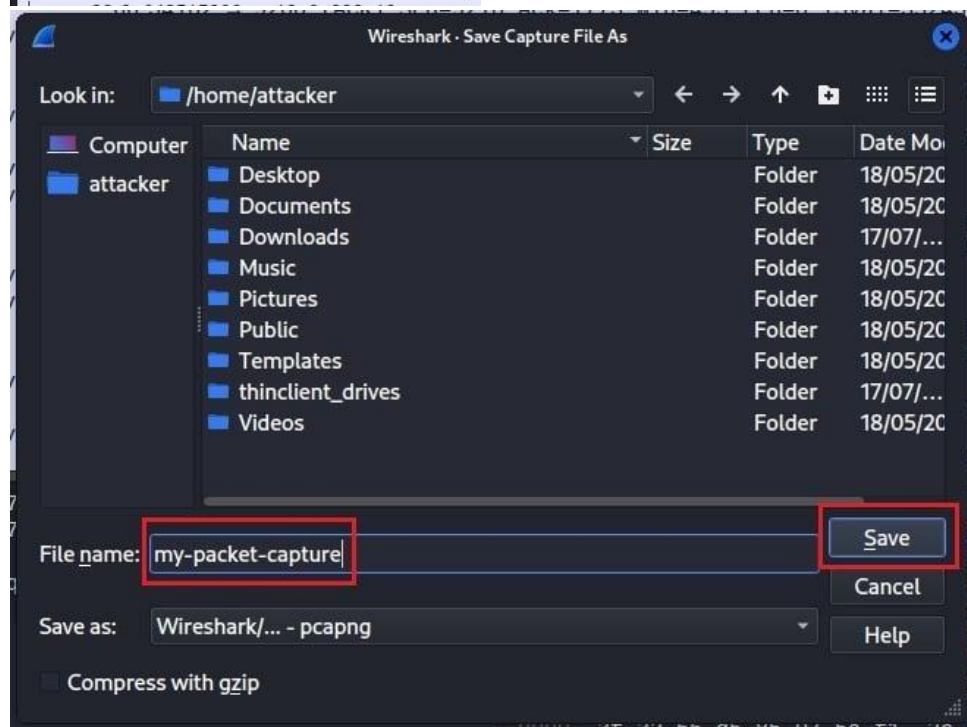
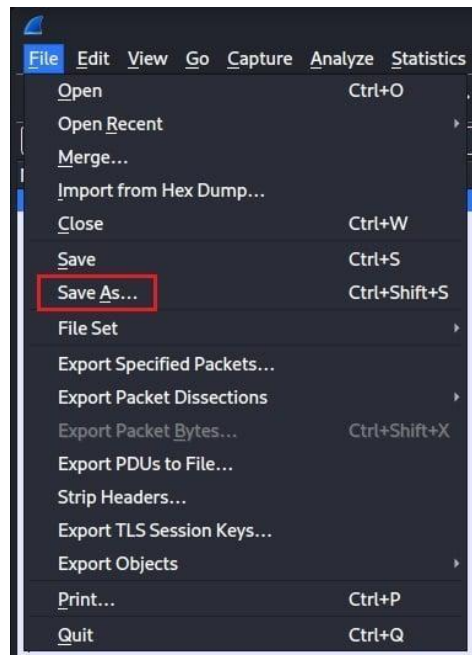
To stop capture packets, click on the red Stop Capture button.



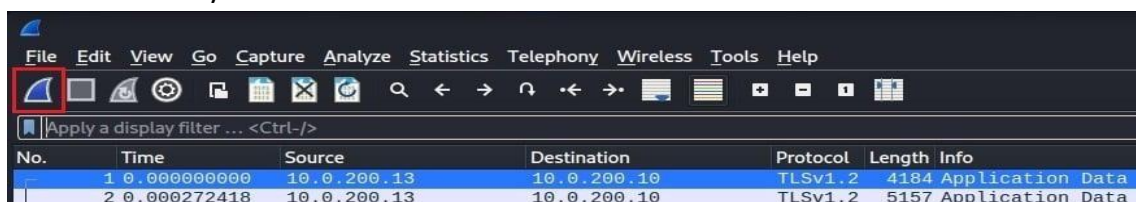
This will save the packet capture into a temporary file and allow you to perform your analysis.



To save your data into a specific file. Select File > Save As, then enter the location and name of the file under which you want to save this data.



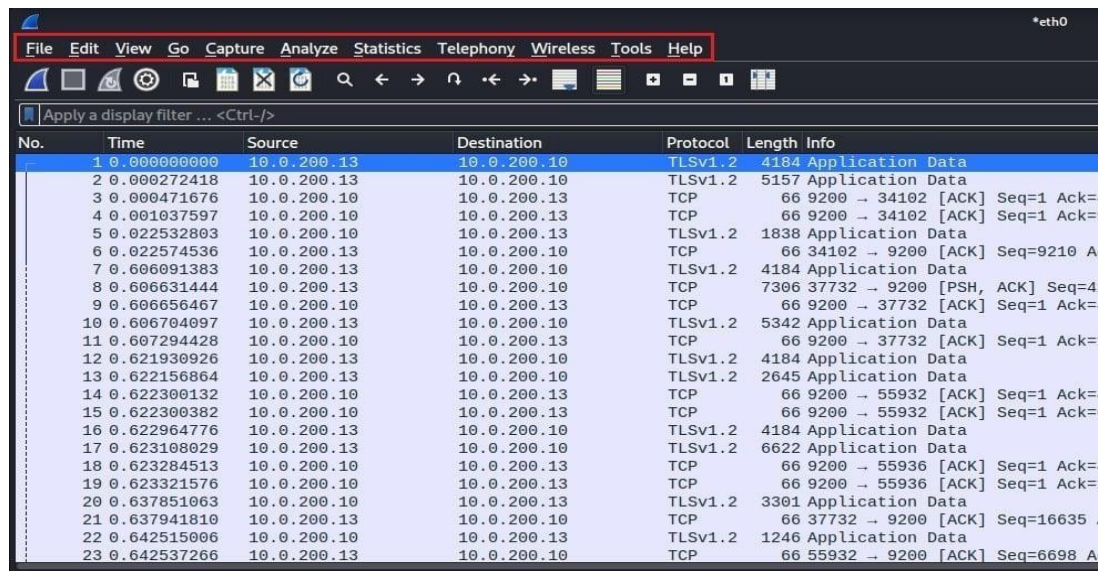
To start a new packet capture, select the Start Capture button. Any unsaved capture data will be automatically deleted.



Understanding the Interface

Once you have captured network traffic or imported a saved packet capture file, you will be greeted by Wireshark's default interface. This interface is comprised of four components.

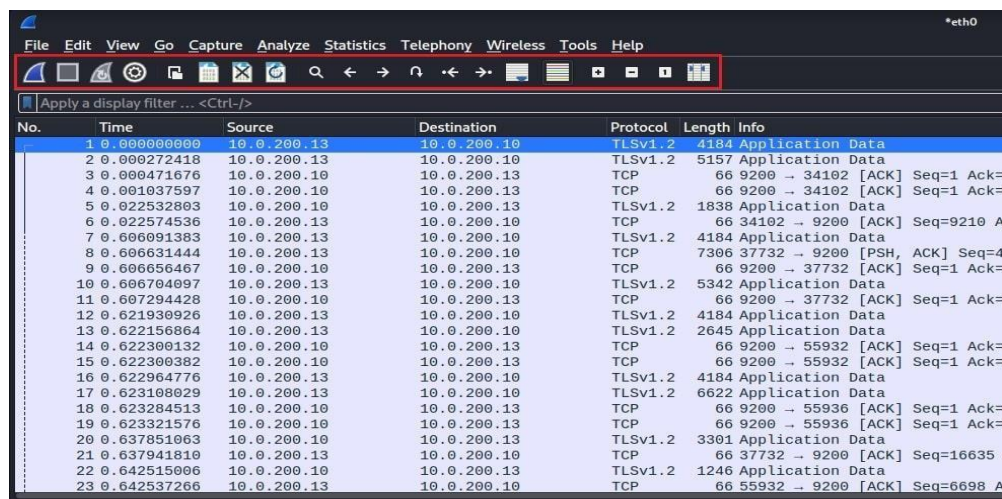
The menu bar



Wireshark's menu bar is located at the top of its interface. It consists of several menus you can use to take advantage of Wireshark's powerful features:

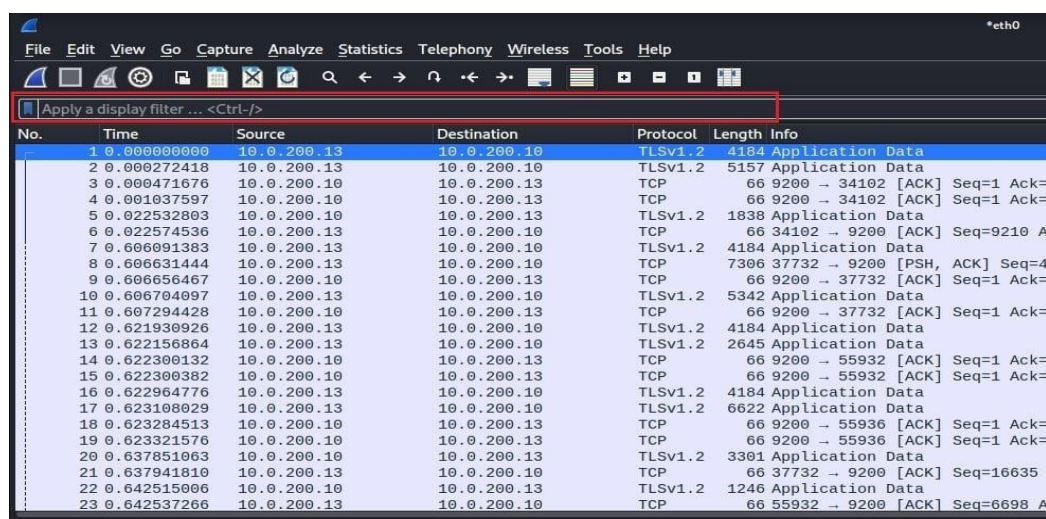
- File: To can open and save files, as well as export packets, TLS session keys, and objects.
- Edit: To modify captured packets.
- View: To customize Wireshark's interface.
- Go: For packet navigation.
- Capture: To control starting, stopping, and configuring packet captures.
- Analyze: Wireshark's tools for analyzing network packets.
- Statistics: For applying statistical analysis to network traffic.
- Telephony: For analyzing Voice over IP (VoIP) traffic.
- Wireless: For analyzing Bluetooth traffic.
- Tools: Additional custom tools written in Lua.
- Help: Links to Wireshark's documentation.

The main toolbar



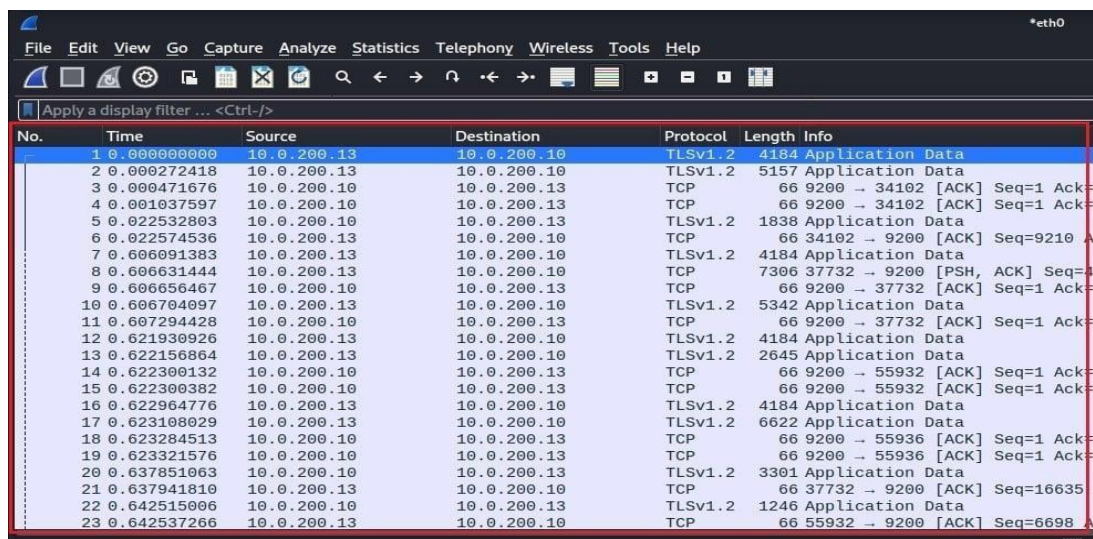
Below the menu bar is Wireshark's main toolbar. These shortcut buttons let you start/stop your packet capture, adjust your capture options, open/close capture files, search for packets, and resize interface panes.

The display filter bar



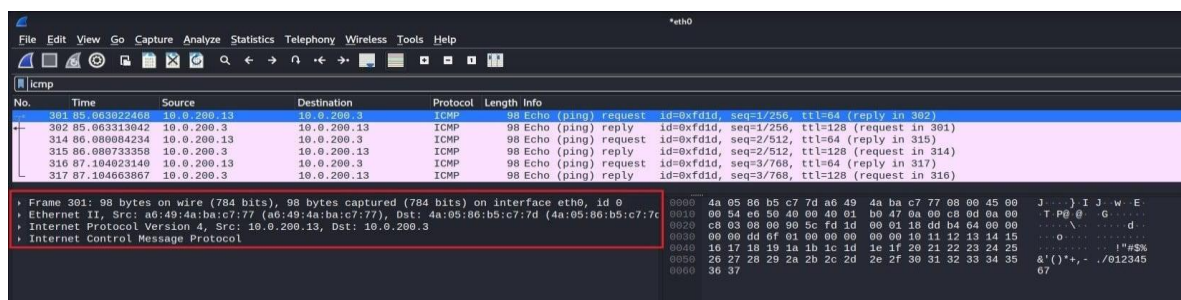
Next down is Wireshark's display filter bar. This allows you to filter the network traffic you have captured based on protocol, source/destination IP address, port number, MAC address, packet type (e.g., ICMP, ARP, etc.), packet length, time range, and packet content. You will see this in action later.

The packet list pane



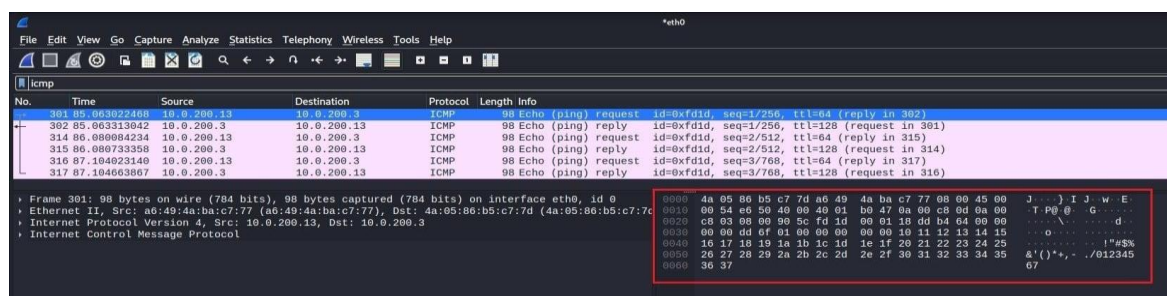
This is the main area of Wireshark's interface. It summarizes the key details of each packet captured, which you can scroll through or sort using the various columns. You can also right-click on an individual packet to perform various actions, such as marking a packet, applying filters, or following a packet stream.

The packet details pane



Once you select a packet, the packet detail pane will populate. This includes the specific network packet details broken down based on TCP/IP network layer. You will see how to inspect individual packets later.

The packet bytes pane



To the right of the packet details pane is the packet bytes pane. This shows the corresponding bytes Wireshark extracted from the wire to reconstruct the network packet data. These bytes are shown in hexadecimal format and are highlighted when you select information within the

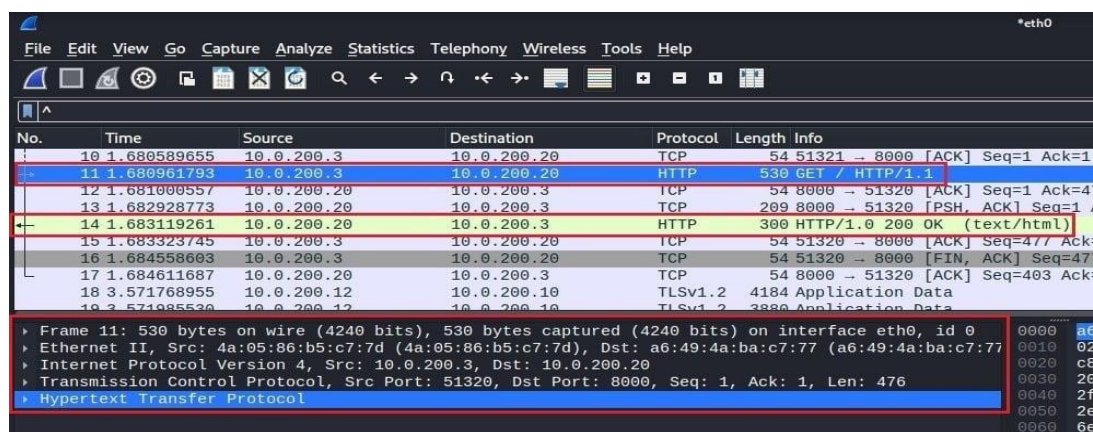
packet details pane. This pane is often used for troubleshooting when Wireshark cannot parse network data.

Analyzing Captured Traffic

You can take several actions in Wireshark to analyze the network traffic you have captured. The first of these is examining the individual packets captured.

Examining Packets

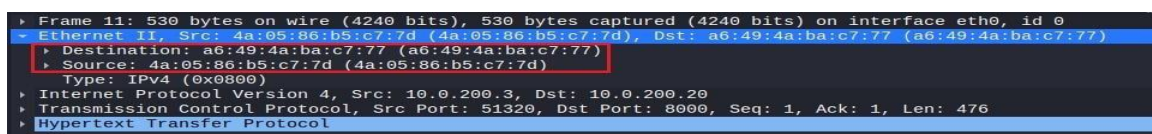
To examine a packet, select it within the packet list pane. This will populate the packet details pane with that packet's network information.



Here packet number 11 has been selected. You see a right-facing arrow indicating a request in Wireshark's packet list pane and a left-facing arrow indicating a response several packets down (packet 14). This is a classic HTTP request and response network conversation.

You will also see the packet details pane populated with network information about packet number 11, including information at the link, Internet, transport, and TCP/IP stack application layer.

To start, you can analyze the network information at the link layer. This includes the source and destination MAC addresses of the two communicating devices and the type of Internet protocol used (IPv4 or IPv6).

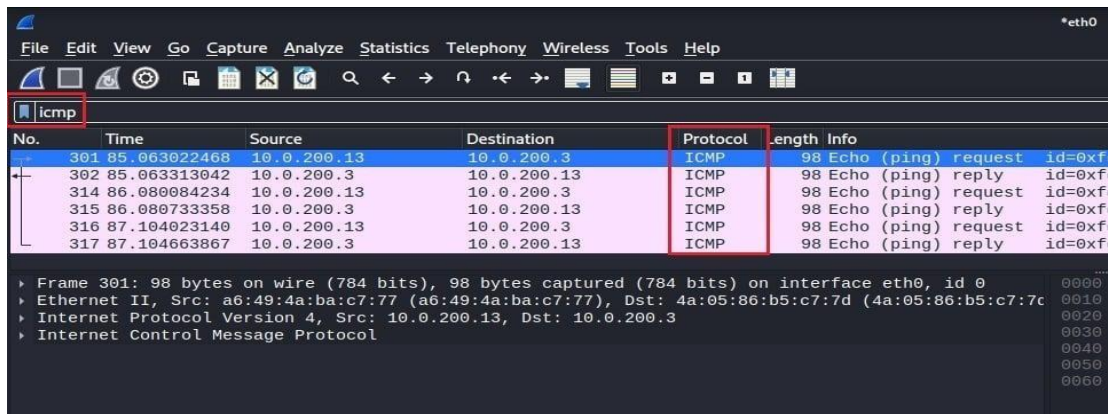


You can then move on to the information included within the Internet layer. This example contains information specific to the IPv4 protocol, such as IP flags, the source and destination IP address, Time to Live (TTL), the transport protocol encapsulated within this packet (e.g., TCP), and other header information.

Refining your view with display filters

You must sift through thousands of network packets when analyzing traffic. To make this task more efficient, Wireshark has built-in display filters that you can use to narrow down the packets displayed in its packet list pane.

To use these filters, enter them into Wireshark's display filter bar (below the main toolbar).



In this screenshot, the filter icmp has been used. This filter will only display packets that use the Internet Control Message Protocol (ICMP) as their network-layer protocol. Now you can analyze all relevant packets without searching through thousands of irrelevant ones. Wireshark will show you how many packets have been filtered out by your display filter at the bottom right of its interface.

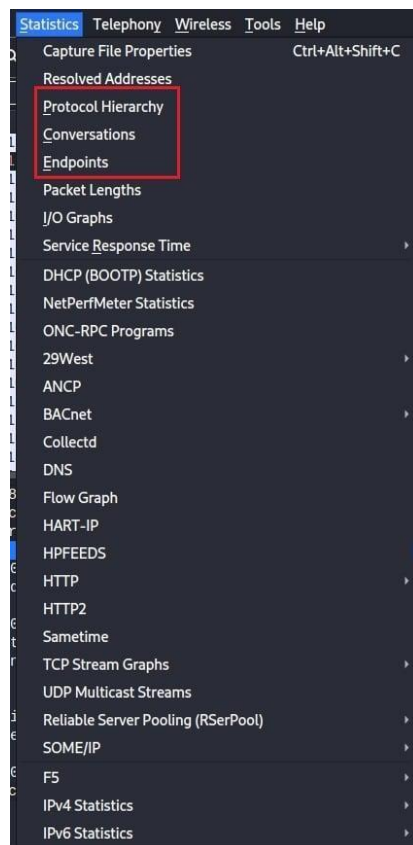
Packets: 319 - Displayed: 6 (1.9%) - Dropped: 0 (0.0%)

You should use display filters to refine your view and speed up your network analysis. To learn more about Wireshark's display filter syntax, look at [Wireshark Cheat Sheet: All the Commands, Filters & Syntax](#). This will show you how to filter by IP address, port number, protocol, and more!

Performing statistical analysis

Aside from analyzing individual network packets, Wireshark also has a powerful statistical analysis feature that lets you quickly summarise your network traffic.

Select the Statistic menu from Wireshark's main menu bar. This will provide options for showing summary information about the protocols being used, the endpoints communicating, and the network conversations between those endpoints.



Select the Protocol Hierarchy option to discover the protocols used within your captured network traffic. This will generate a window summarizing the network protocols used at each TCP/IP stack layer.

Wireshark - Protocol Hierarchy Statistics - eth0

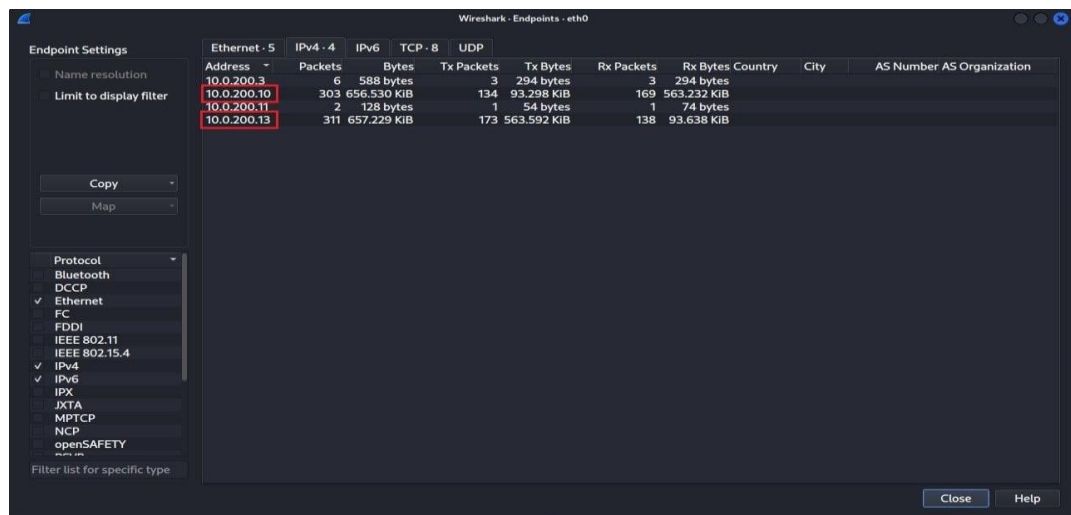
Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU/s
Frame	100.0	319	100.0	673339	60 k	0	0	0	319
Ethernet	100.0	319	0.7	4466	398	0	0	0	319
Internet Protocol Version 4	97.5	311	0.9	6220	555	0	0	0	311
Transmission Control Protocol	95.6	305	98.3	662045	59 k	165	168900	15 k	305
Transport Layer Security	43.9	140	96.9	652289	58 k	140	553853	49 k	146
Internet Control Message Protocol	1.9	6	0.1	384	34	6	384	34	6
Address Resolution Protocol	2.5	8	0.0	224	19	8	224	19	8

No display filter.

Close Copy Help

Select the Endpoints option to find out what endpoints are present in your network traffic. This will list all the unique endpoint devices communicating within the network packets you have captured based on Ethernet address, IP address, and TCP/UDP port.

In the example below, the IPv4 tab has been selected. This shows that four devices are communicating in this packet capture, with two being the primary exchangers of data.



Select the Conversations option to delve into the network conversations within your packet capture. This will show you the network traffic traveling between endpoints. Here you can see a lot of data is traveling between 10.0.200.13 and 10.0.200.10, while very little is between 10.0.200.13 and 10.0.200.11.

Protocol	Address A	Address B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
IPv4	10.0.200.13	10.0.200.3	6	588 bytes	3	294 bytes	3	294 bytes	85.063022	2.0416	1,152 bits/s	1,152 bits/s
	10.0.200.13	10.0.200.10	303	656.530 KiB	169	563.232 KiB	134	93.298 KiB	0.000000	85.7818	53 kbps	8,909 bits/s
	10.0.200.13	10.0.200.11	2	128 bytes	1	74 bytes	1	54 bytes	25.462631	0.0004		

You can use Wireshark's statistical analysis feature to quickly identify outliers within your packet capture and refine your search using display filters to analyze these outliers.

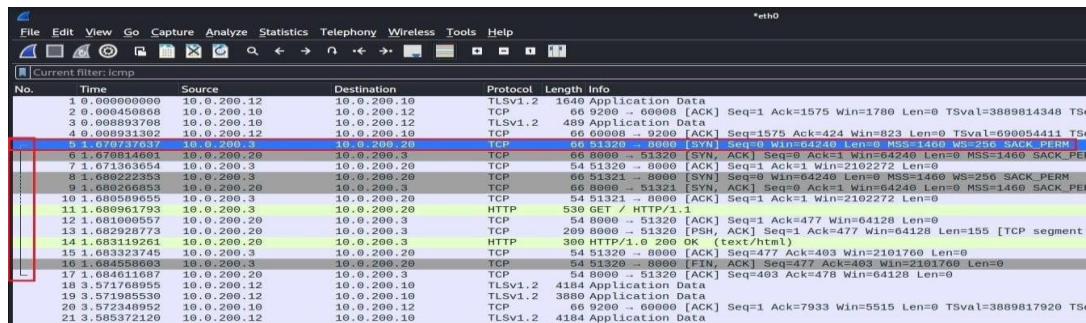
Techniques

Now that you have a basic understanding of capturing and analyzing traffic with Wireshark let's look at how you can use this powerful network protocol analyzer in the real world. The following are three use cases you will likely encounter whether you are on the red or blue team in cyber security.

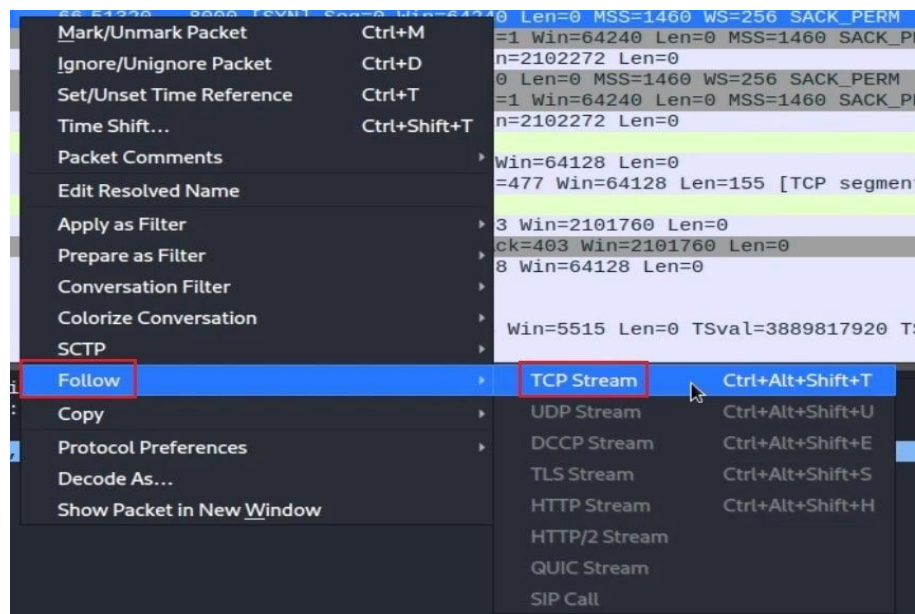
Following TCP Streams

One of the most common use cases for Wireshark is following TCP streams. By following this stream, you can see the conversation between the two devices and the data they exchanged. This is useful for troubleshooting network issues or discovering hidden information.

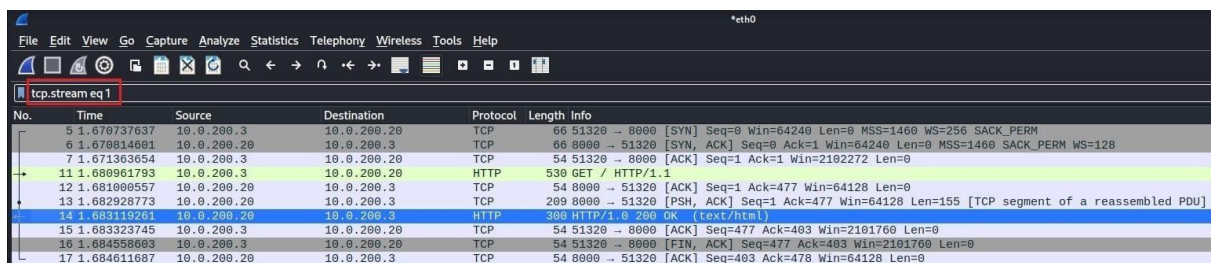
To follow a TCP stream in Wireshark, right-click on a packet whose TCP stream you want to follow.



Here packet number 5 has been selected. It is a TCP connect request (TCP SYN flag) sent to port 8000 on machine 10.0.200.20 and is the start of the [TCP three-way handshake](#). A drop-down arrow on the left of the packet list pane indicates the entire network conversation down to packet number 17. To follow this TCP stream, right-click on packet number 5 and select Follow > TCP Stream.



Wireshark will automatically apply a display filter that only shows packets from this TCP stream.



Wireshark will also generate a summary view of the TCP stream in a new window. In this case, it highlights the HTTP messages sent between a client and server.

Wireshark - Follow TCP Stream (tcp.stream eq 1) - eth0

```
GET / HTTP/1.1
Host: 10.0.200.20:8000
Connection: keep-alive
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36 Edg/113.0.1774.42
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: en-GB,en;q=0.9,en-US;q=0.8

HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.11.2
Date: Mon, 17 Jul 2023 07:22:26 GMT
Content-type: text/html; charset=utf-8
Content-Length: 246

<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href="stationx-logo.png">stationx-logo.png</a></li>
</ul>
<hr>
</body>
</html>
```

Entire conversation (877 bytes) | Show data as ASCII | Stream 1

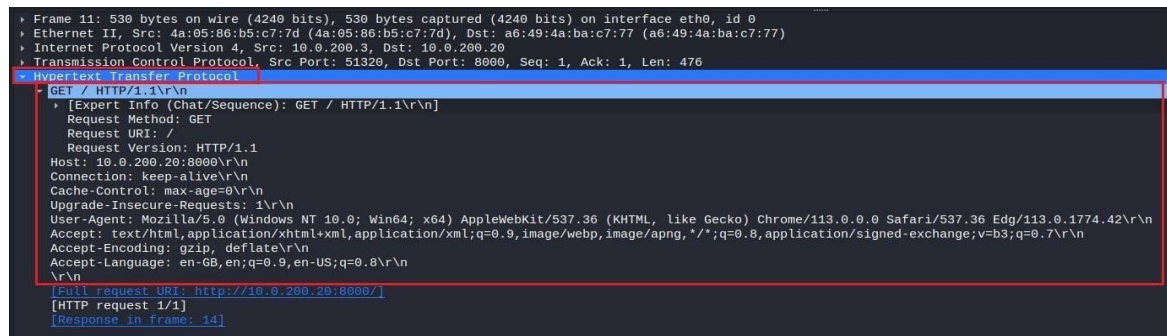
Find: | Filter Out This Stream | Print | Save as... | Back | Close | Help

Following the IP layer is the transport layer. This network protocol is responsible for the end-to-end data delivery between hosts and will be either UDP or TCP. It includes the source and destination port of the segment, TCP-specific header flags, the size of the encapsulated application message, and other header information.

```
> Frame 11: 530 bytes on wire (4240 bits), 530 bytes captured (4240 bits) on interface eth0, id 0
> Ethernet II, Src: 4a:05:86:b5:c7:7d (4a:05:86:b5:c7:7d), Dst: a6:49:4a:ba:c7:77 (a6:49:4a:ba:c7:77)
> Internet Protocol Version 4, Src: 10.0.200.3, Dst: 10.0.200.20
> Transmission Control Protocol, Src Port: 51320, Dst Port: 8000, Seq: 1, Ack: 1, Len: 476
  Source Port: 51320
  Destination Port: 8000
  [Stream index: 1]
  [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 476]
  Sequence Number: 1 (relative sequence number)
  Sequence Number (raw): 3261474857
  [Next Sequence Number: 477 (relative sequence number)]
  Acknowledgment Number: 1 (relative ack number)
  Acknowledgment number (raw): 1932156097
  0101 ..... = Header Length: 20 bytes (5)
  > Flags: 0x018 (PSH, ACK)
    Window: 8212
    [Calculated window size: 2102272]
    [Window size scaling factor: 256]
    Checksum: 0xc7ce [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > [Timestamps]
  > [SEQ/ACK analysis]
    TCP payload (476 bytes)
> Hypertext Transfer Protocol
```

Finally, the packet details pane will show you network information about the packet's application protocol (if an application protocol is used). In this case, HTTP is the application protocol, so you can see information about the request method, request URI, and other

common HTTP headers. This will be where you find most of the network information you want to analyze.

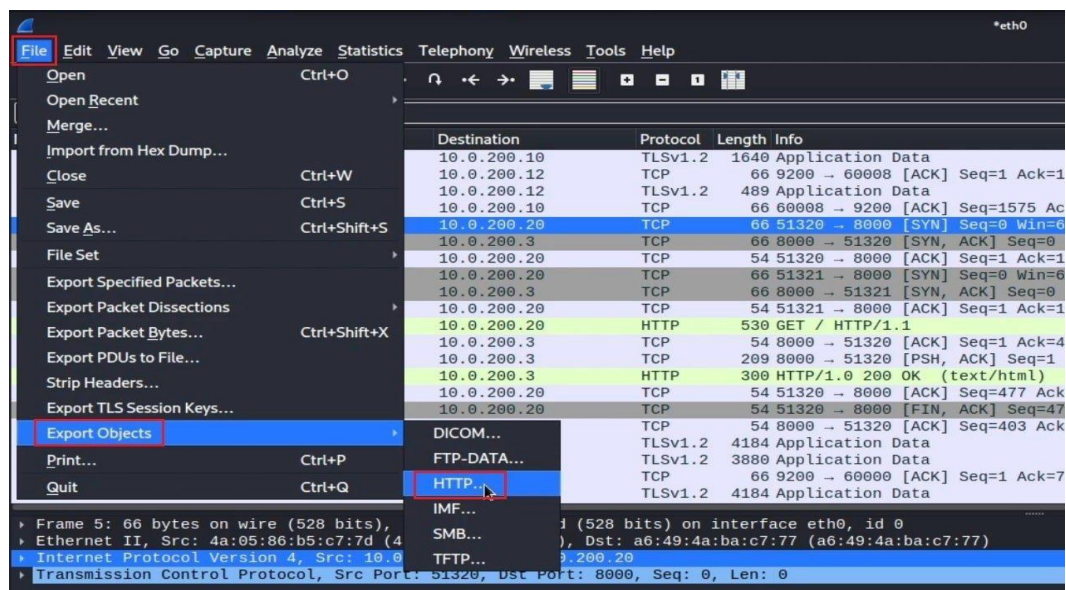


```
Frame 11: 530 bytes on wire (4240 bits), 530 bytes captured (4240 bits) on interface eth0, id 0
Ethernet II, Src: 4a:05:86:b5:c7:7d (4a:05:86:b5:c7:7d), Dst: a6:49:4a:ba:c7:77 (a6:49:4a:ba:c7:77)
Internet Protocol Version 4, Src: 10.0.200.3, Dst: 10.0.200.20
Transmission Control Protocol, Src Port: 51320, Dst Port: 8000, Seq: 1, Ack: 1, Len: 476
Hypertext Transfer Protocol
  GET / HTTP/1.1\r\n
    [Expert Info (Chat/Sequence): GET / HTTP/1.1\r\n]
    Request Method: GET
    Request URI: /
    Request Version: HTTP/1.1
    Host: 10.0.200.20:8000\r\n
    Connection: keep-alive\r\n
    Cache-Control: max-age=0\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36 Edg/113.0.1774.42\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: en-GB,en;q=0.9,en-US;q=0.8\r\n
    \r\n
  [Full request URI: http://10.0.200.20:8000/]
  [HTTP request 1/1]
  [Response in frame: 14]
```

Extracting Files

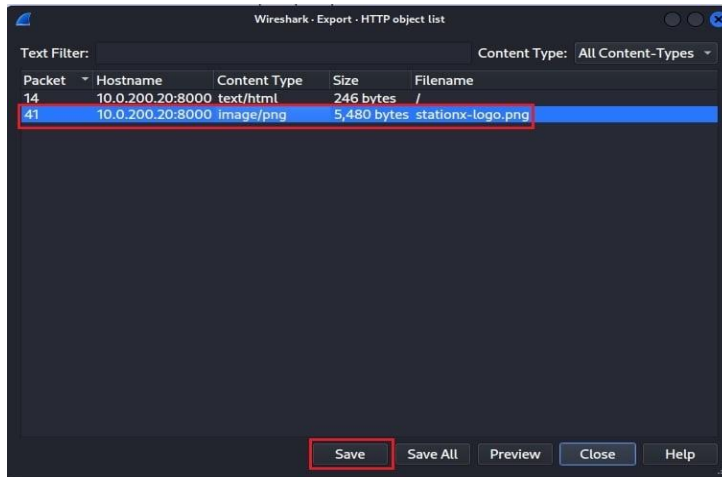
If you are on the red team, you often use Wireshark to extract sensitive information, such as usernames, passwords, and files. Wireshark can reconstruct files from the capture packets. This feature is incredibly useful as it allows you to extract various file types from network packets.

To extract files in Wireshark, select File > Export Objects. You can then select the object type you want to export based on the protocol used to transmit said object. In this case, an image was transferred using HTTP.

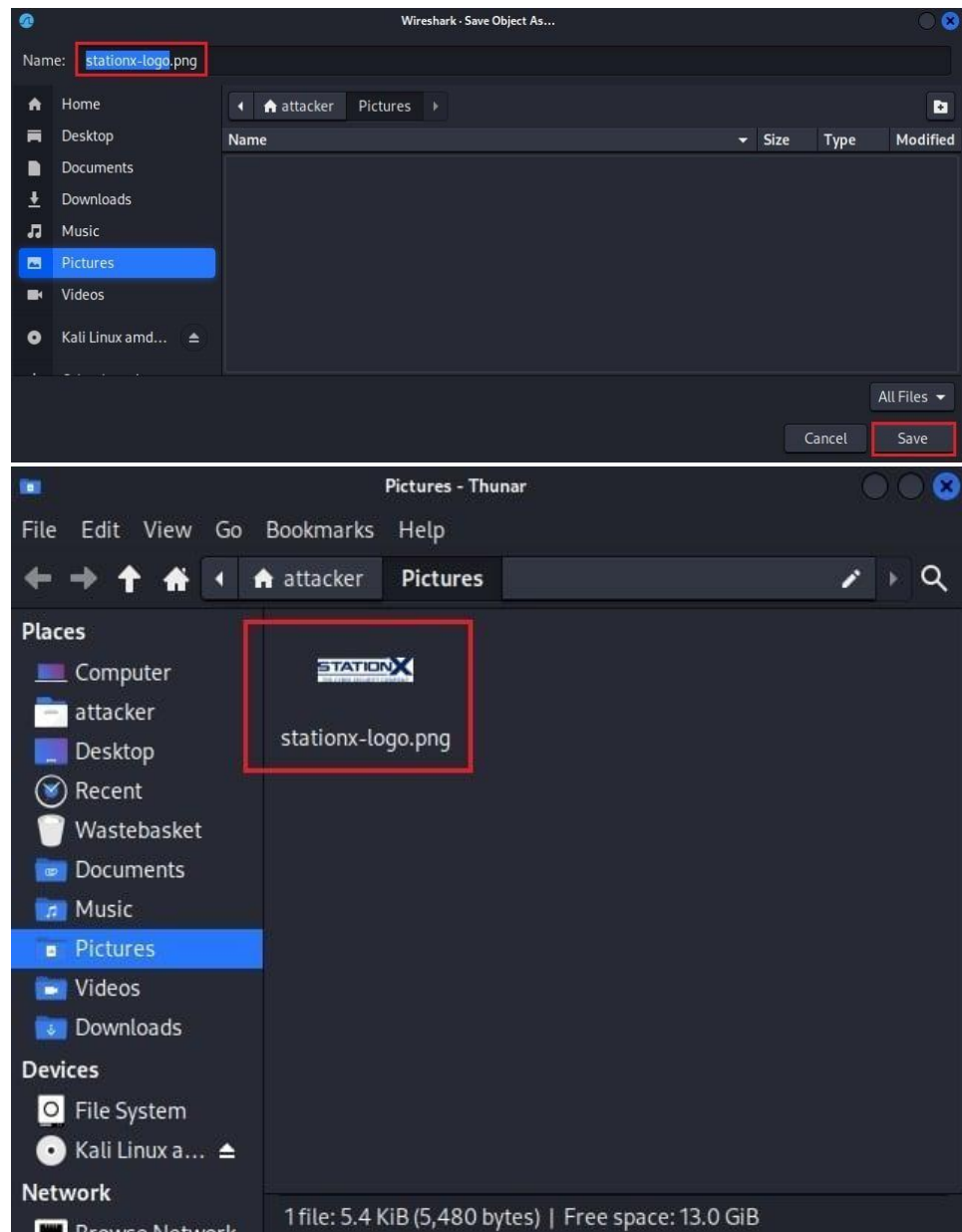


Selecting HTTP opens a new window populated by all the objects transferred using HTTP in the packet capture file. There are two objects; a static HTML page and a PNG image.

To extract the image, select the packet (41) and click Save.



Now choose a location to save the image file to, and you can open the image in your favorite image viewer.

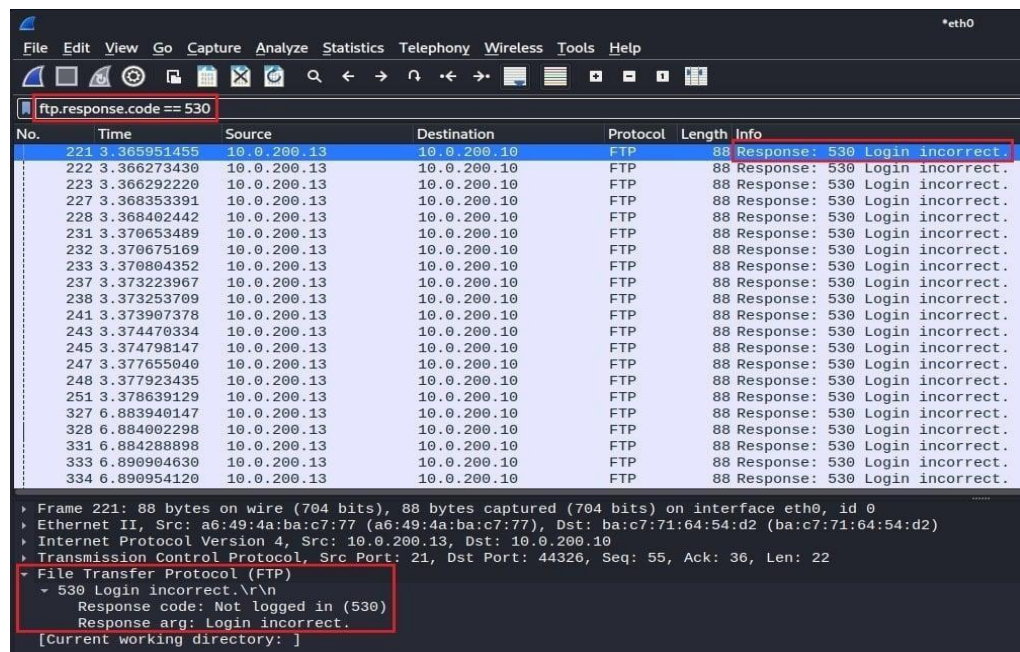


Using Wireshark, you can extract compressed files, executables, images, videos, email attachments, and other binary file types.

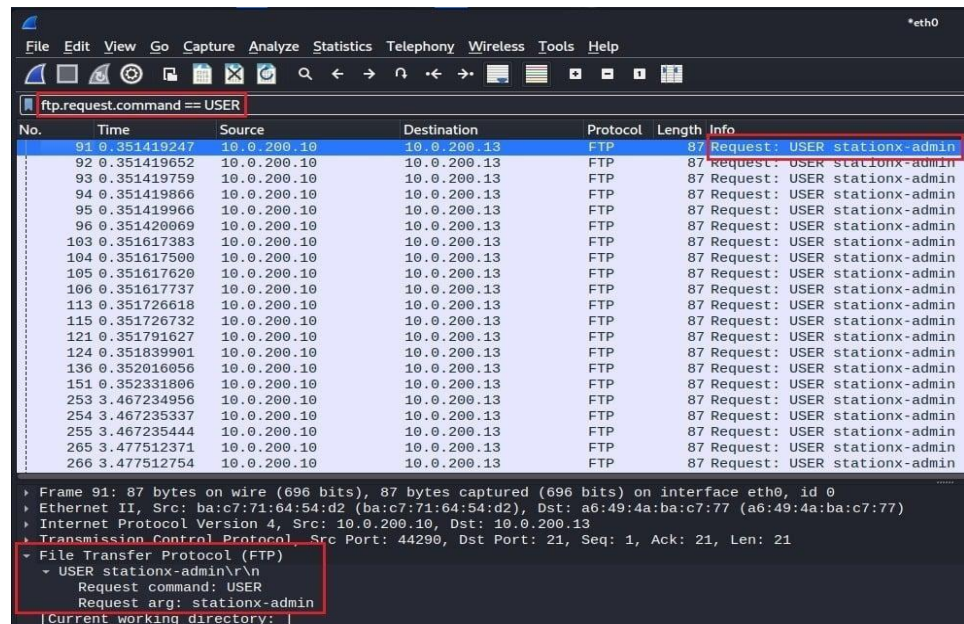
Identifying Attacks

If you are on the blue team, you will likely use Wireshark to identify cyber attacks by analyzing network traffic. This includes brute force attacks, port scans, and data exfiltration. Let's look at identifying an FTP brute force attack in Wireshark. The following packet capture was performed after running the [Hydra online cracking tool](#) against an FTP server.

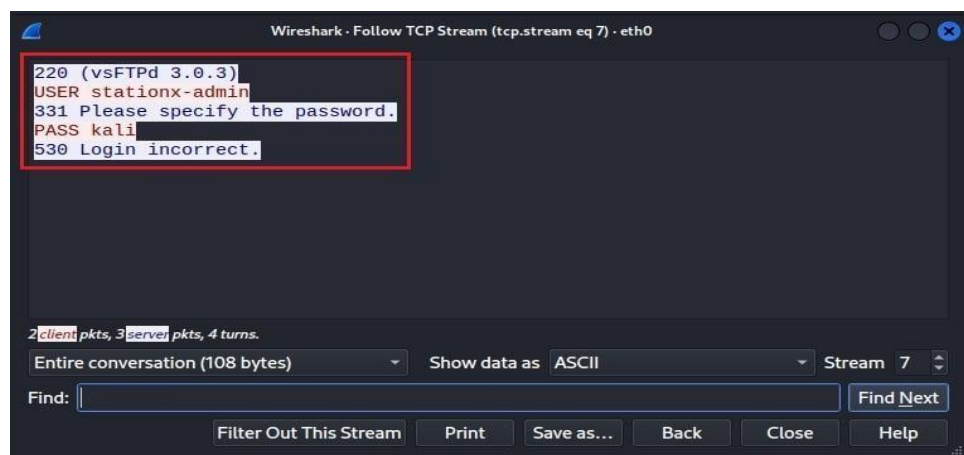
To see the FTP traffic, you can use the Wireshark display filter `ftp.response.code == 530`. This is the FTP response code for "Not logged in" and indicates an authentication failure. Seeing several of these errors with the same username but a different password indicates an FTP brute force attack.



To find out if they belong to the same user, change the display filter to `ftp.request.command == USER`. This will show you all the FTP login requests where a username was specified.

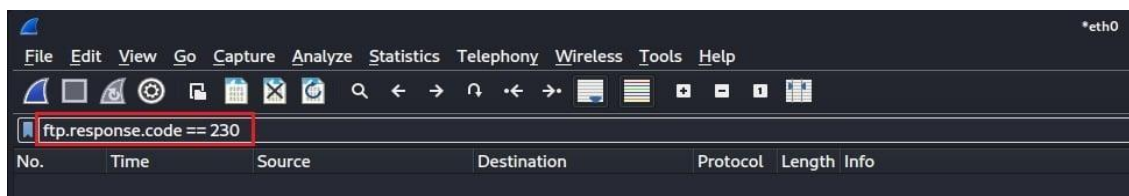


Next, to determine if different passwords are being tried, right-click on a network packet, and select Follow > TCP Stream.



Here the same user tries multiple passwords to log in, indicating a password spray attack against the user stationx-admin.

To determine if any attempts were successful, use the display filter ftp.response.code == 230 to look for "User logged in, proceed."



No login attempts were successful!

You can find out what cyber security team you should join in the [Red Team vs Blue Team: Which Is the Best Choice for You?](#)

Output:

The image shows a Wireshark packet capture window titled "Ethernet". The packet list on the left shows a series of packets from 10.111.97.53 to 8.8.8.8. The packet details pane on the right shows the structure of a packet, including Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and Domain Name System (query). The packet bytes pane on the right shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
22059	22.042956700	10.111.97.53	8.8.8.8	TCP	66	63870 → 53 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
22060	22.068794600	8.8.8.8	10.111.97.53	TCP	66	53 → 63870 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
22067	22.068716400	10.111.97.53	8.8.8.8	TCP	54	63870 → 53 [ACK] Seq=1 Ack=1 Win=65280 Len=0
22068	22.068759200	10.111.97.53	8.8.8.8	TCP	56	63870 → 53 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=2 [TCP PDU reassembled in 22089]
22089	22.068767600	10.111.97.53	8.8.8.8	DNS	83	Standard query 0xe74e A youtube.com
22304	22.094506200	8.8.8.8	10.111.97.53	TCP	60	53 → 63870 [ACK] Seq=1 Ack=3 Win=269824 Len=0
22305	22.094506500	8.8.8.8	10.111.97.53	TCP	60	53 → 63870 [ACK] Seq=1 Ack=32 Win=269824 Len=0
22350	22.095730000	8.8.8.8	10.111.97.53	DNS	149	Standard query response 0xe74e A youtube.com A 192.178.211.91 A 192.178.211.136 A 192.178.211.190 A 192.178.211.93
22393	22.096428100	10.111.97.53	8.8.8.8	TCP	54	63870 → 53 [FIN, ACK] Seq=32 Ack=96 Win=65280 Len=0
22847	22.122020900	8.8.8.8	10.111.97.53	TCP	60	53 → 63870 [FIN, ACK] Seq=96 Ack=33 Win=269824 Len=0
22848	22.122040500	10.111.97.53	8.8.8.8	TCP	54	63870 → 53 [ACK] Seq=33 Ack=97 Win=65280 Len=0

Frame 22089: Packet, 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface \Device\NPF... (DCEI1)

Ethernet II, Src: EliteGroupCo_f2:72:51 (88:ae:dd:f2:72:51), Dst: CiscoMeraki_4f:00:01 (00:18:0a:4f:00:01)

Internet Protocol Version 4, Src: 10.111.97.53, Dst: 8.8.8.8

Transmission Control Protocol, Src Port: 63870, Dst Port: 53, Seq: 3, Ack: 1, Len: 29

[2 Reassembled TCP Segments (31 bytes): #22088(2), #22089(29)]

Domain Name System (query)

Packet (83 bytes) Reassembled TCP (31 bytes)

Packets: 25838 - Displayed: 11 (0.0%) - Dropped: 0 (0.0%) Profile: Default

The image shows a Wireshark packet capture window titled "Ethernet". The packet list on the left shows a series of packets from 10.111.97.53 to 8.8.8.8. The packet details pane on the right shows the structure of a packet, including Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and Domain Name System (query). The packet bytes pane on the right shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
22059	22.042956700	10.111.97.53	8.8.8.8	TCP	66	63870 → 53 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
22060	22.068794600	8.8.8.8	10.111.97.53	TCP	66	53 → 63870 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256
22067	22.068716400	10.111.97.53	8.8.8.8	TCP	54	63870 → 53 [ACK] Seq=1 Ack=1 Win=65280 Len=0
22068	22.068759200	10.111.97.53	8.8.8.8	TCP	56	63870 → 53 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=2 [TCP PDU reassembled in 22089]
22089	22.068767600	10.111.97.53	8.8.8.8	DNS	83	Standard query 0xe74e A youtube.com
22304	22.094506200	8.8.8.8	10.111.97.53	TCP	60	53 → 63870 [ACK] Seq=1 Ack=3 Win=269824 Len=0
22305	22.094506500	8.8.8.8	10.111.97.53	TCP	60	53 → 63870 [ACK] Seq=1 Ack=32 Win=269824 Len=0
22350	22.095730000	8.8.8.8	10.111.97.53	DNS	149	Standard query response 0xe74e A youtube.com A 192.178.211.91 A 192.178.211.136 A 192.178.211.190 A 192.178.211.93
22393	22.096428100	10.111.97.53	8.8.8.8	TCP	54	63870 → 53 [FIN, ACK] Seq=32 Ack=96 Win=65280 Len=0
22847	22.122020900	8.8.8.8	10.111.97.53	TCP	60	53 → 63870 [FIN, ACK] Seq=96 Ack=33 Win=269824 Len=0
22848	22.122040500	10.111.97.53	8.8.8.8	TCP	54	63870 → 53 [ACK] Seq=33 Ack=97 Win=65280 Len=0

Frame 22089: Packet, 83 bytes on wire (664 bits), 83 bytes captured (664 bits) on interface \Device\NPF... (DCEI1)

Ethernet II, Src: EliteGroupCo_f2:72:51 (88:ae:dd:f2:72:51), Dst: CiscoMeraki_4f:00:01 (00:18:0a:4f:00:01)

Internet Protocol Version 4, Src: 10.111.97.53, Dst: 8.8.8.8

Transmission Control Protocol, Src Port: 63870, Dst Port: 53, Seq: 3, Ack: 1, Len: 29

[2 Reassembled TCP Segments (31 bytes): #22088(2), #22089(29)]

Domain Name System (query)

Packet (83 bytes) Reassembled TCP (31 bytes)

Packets: 25838 - Displayed: 11 (0.0%) - Dropped: 0 (0.0%) Profile: Default

Conclusion

Wireshark is a powerful open-source network protocol analyzer that turns bytes on the wire into network traffic you can analyze. Its simple-to-use interface provides an overview of your capture traffic in the list pane and specific information about each packet in the details pane.

You have seen how to examine network packets in granular detail, refine your view using Wireshark's display filters, and summarize network traffic with statistical analysis tools. These skills allow you to use Wireshark in the real world for following TCP streams to uncover conversations, extracting files, and identifying cyber attacks